

Nivel 1

Ejercicio 1

A partir de los documentos adjuntos (estructura_dades i dades_introduir), importa las dos tablas. Muestra las principales características del esquema creado y explica las diferentes tablas y variables que existen. Asegúrate de incluir un diagrama que ilustre la relación entre las distintas tablas y variables.

Paso 1. Las base de datos cargada y los datos de las tablas introducidos

The screenshot shows a database interface with a sidebar on the left containing a tree view of the database schema. The 'company' table is selected under the 'transactions' folder. The main area displays the 'SELECT * FROM company;' query and a 'Result Grid' showing the table's data. The table has columns: id, company_name, phone, email, country, and website. The data includes entries for Fusce Corp., Convallis In Incorporated, Ante Iaculis Nec Foundation, Donec Ltd, and Sed Nunc Ltd.

id	company_name	phone	email	country	website
b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United States	https://pinterest.com/sub/cars
b-2234	Convallis In Incorporated	06 66 57 29 50	mauris.ut@aol.couk	Germany	https://cnn.com/user/110
b-2238	Ante Iaculis Nec Foundation	08 23 04 99 53	sed.dictum.proin@outlook.ca	New Zealand	https://netflix.com/settings
b-2242	Donec Ltd	01 25 51 37 37	at.iaculis@hotmail.couk	Norway	https://nytimes.com/user/110
b-2246	Sed Nunc Ltd	02 62 64 73 48	nibh@yahoo.org	United Kingdom	https://cnn.com/one

10... 11:29:24 SELECT * FROM company 100 row(s) returned

The screenshot shows the same database interface with the 'transaction' table selected. The main area displays the 'SELECT * FROM transaction;' query and a 'Result Grid' showing the table's data. The table has columns: id, credit_card..., company_id, user_id, lat, longitude, timestamp, amount, and declined. The data includes entries for various transactions, such as those with IDs 00045D6B-ED2E-4F2F-8186-CEE074D875D0 and 000481C3-1C26-4FEF-83A0-4CD0EB004BBB.

id	credit_card...	company_id	user_id	lat	longitude	timestamp	amount	declined
00045D6B-ED2E-4F2F-8186-CEE074D875D0	CcS-6699	b-2390	2118	29.7573	-95.3796	2020-07-14 15:37:45	326.01	0
000481C3-1C26-4FEF-83A0-4CD0EB004BBB	CcS-6696	b-2230	2115	53.5489	-113.503	2017-09-04 19:44:53	161.60	0
00051AA4-9CBE-4268-B070-C38062A1B3E2	CcS-7606	b-2266	3025	52.2084	5.69081	2017-01-05 18:19:25	148.91	0
0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	CcU-3358	b-2598	215	53.5535	-113.499	2023-09-23 04:51:43	294.59	0
0009A151-9BCF-4E31-9053-A468FF77FAAB	CcS-7509	b-2546	2928	51.9362	5.34265	2023-12-31 00:06:36	383.63	0

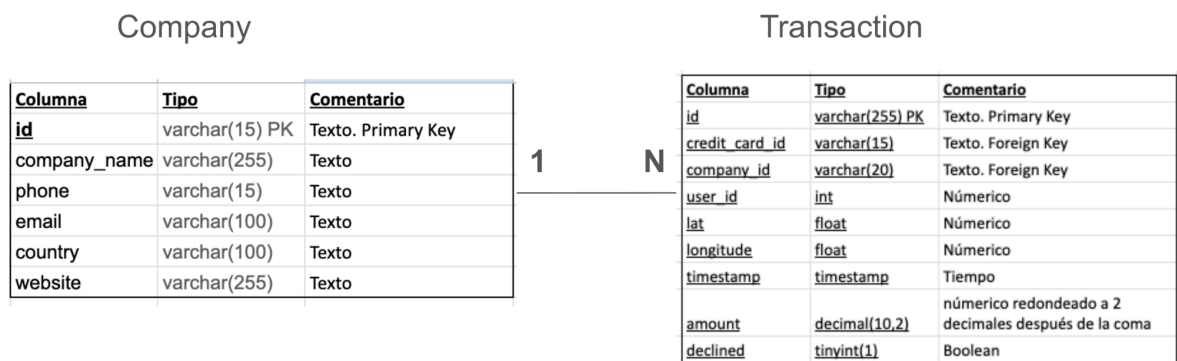
10... 11:30:01 SELECT * FROM transaction 100000 row(s) returned

Paso 2.

Abajo está el esquema de las tablas con las columnas creadas. La relación entre las tablas es: Company (1) - Transaction (N). Están relacionadas a través de la columnas [company.id](#) (es una primary key en la tabla Company) y Transaction.company_id (es una foreign key en la tabla Transaction).

La tabla Company contiene la información sobre las empresas que están guardadas en las variables textuales (tipo varchar).

La tabla Transaction contiene la información de cada transacción que incluye los datos de la tarjeta de crédito, la empresa, el usuario, donde y cuando se realiza la transacción, el importe y si la transacción ha sido rechazada o no.



Ejercicio 2

Utilizando JOIN realizarás las siguientes consultas:

1. Listado de los países que están generando ventas.

```
1  ## Listado de los países que están generando ventas.
2  • SELECT company.country FROM transactions.company
3    JOIN transactions.transaction
4    ON company.id = transaction.company_id
5    GROUP BY company.country;
6
```

Result Grid



Filter Rows:

country
Germany
Australia
United States
New Zealand
Norway

10... 11:27:11 SELECT company.country FROM transactions.company JOIN transactions.transaction ON company.id = transaction.co... 15 row(s) returned

2. Desde cuántos países se generan las ventas.

Desde cuántos países se generan las ventas.

```
SELECT COUNT(DISTINCT company.country) as num_companies FROM transactions.company
JOIN transactions.transaction
ON company.id = transaction.company_id;
```

num_companies
16

SELECT COUNT(DISTINCT company.country) as num_companies FROM transactions.company JOIN transactions.trans... 1 row(s) returned

3. Identifica a la compañía con la mayor media de ventas.
(Como en el ejercicio no pone usar subquery lo hice así. Aunque lo más óptimo hubiera sido hacer una subquery)

```

SELECT company.id, company.company_name, ROUND(AVG(transaction.amount), 2) as avg_sale
FROM transactions.company
JOIN transactions.transaction
ON company.id = transaction.company_id
GROUP BY company.id
ORDER BY avg_sale DESC
LIMIT 1;

```

id	company_name	avg_sale	
b-2222	Ac Fermentum Incorporated	284.87	

```

SELECT company.id, company.company_name, ROUND(AVG(transaction.amount), 2) as avg_sale FROM transactions.c... 1 row(s) returned

```

Ejercicio 3

Utilizando sólo subconsultas (sin utilizar JOIN):

1. Muestra todas las transacciones realizadas por empresas de Alemania.

```
SELECT transaction.id FROM transactions.transaction
WHERE transaction.company_id IN (
    SELECT company.id FROM transactions.company
    WHERE company.country = "Germany"
);
```

id
00687139-48B2-4FFA-8E73-B20376F04AB4
0074F4DD-32F1-4827-8758-55896314623A
00AAB9CD-39D6-4DCB-8A1D-13BE73DC90A9
00BE09D4-6920-47D8-ABE8-325E2269829D
00DA0383-E048-4577-8ED1-3C56C258FF2F
00DD11DE-ED01-4BBD-93A0-174D183A59DF
01449CE0-98E9-4DE5-9810-728C6BA00E6F
0175E8C7-241E-42DA-A8B9-9F246DBF4D2F
01ABDAB8-06E2-4CA0-A131-AEE6FF11B749

10... 12:19:36 SELECT transaction.id FROM transactions.transaction WHERE transaction.company_id IN (SELECT company.id FROM... 13291 row(s) returned

2. Lista las empresas que han realizado transacciones por un importe superior a la media de todas las transacciones. (Son todas las empresas)

```

SELECT company.id, company.company_name FROM transactions.company
WHERE company.id IN (
    SELECT transaction.company_id FROM transactions.transaction
    WHERE transaction.amount > (
        SELECT AVG(transaction.amount) FROM transactions.transaction
    )
);

```

id	company_name
b-2222	Ac Fermentum Incorporated
b-2226	Magna A Neque Industries
b-2230	Fusce Corp.

SELECT company.id, company.company_name FROM transactions.company WHERE company.id IN (SELECT transacti... 100 row(s) returned

3. Eliminarán del sistema las empresas que carecen de transacciones registradas, entrega el listado de estas empresas. (Todas las empresas tienen transacciones)

```

##Eliminarán del sistema las empresas que carecen de transacciones registradas, entrega el listado de estas empresas.
SELECT company.id, company.company_name FROM transactions.company
WHERE company.id NOT IN (
    SELECT transaction.company_id FROM transactions.transaction
);

```

Result Grid		Filter Rows:
id	company_name	
NULL	NULL	

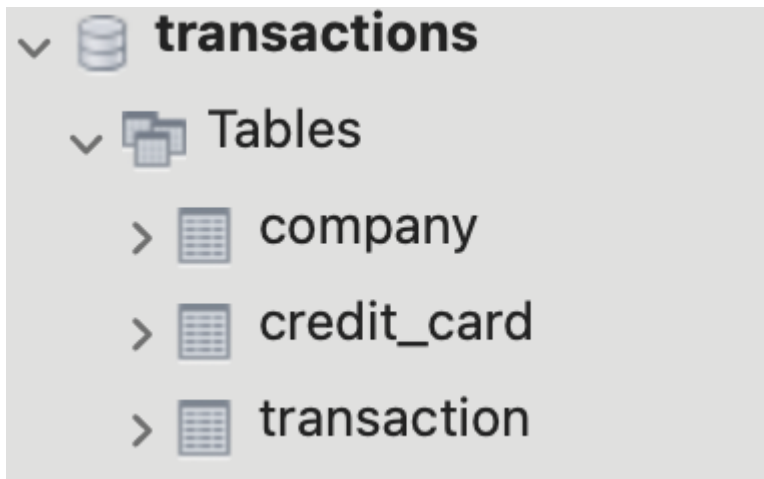
SELECT company.id, company.company_name FROM transactions.company WHERE company.id NOT IN (SELECT tran... 0 row(s) returned

Ejercicio 4

Tu tarea es diseñar y crear una tabla llamada "credit_card" que almacene detalles cruciales sobre las tarjetas de crédito. La nueva tabla debe ser capaz de identificar de forma única cada tarjeta y establecer una relación adecuada con las otras dos tablas ("transaction" y "company"). Después de crear la tabla será necesario que ingreses la información del documento denominado "datos_introducir_credit". Recuerda mostrar el diagrama y realizar una breve descripción del mismo.

Paso 1. La tabla creada

```
CREATE TABLE IF NOT EXISTS credit_card (  
  id VARCHAR(15) PRIMARY KEY,  
  iban VARCHAR(34),  
  pan VARCHAR(19),  
  pin CHAR(4),  
  cvv CHAR(3),  
  expiring_date VARCHAR(10)  
);
```



Paso 2. Los datos cargados

```
1 • select * from credit_card;
```

0% 27:1

Result Grid Filter Rows: Search Edit: Export/

id	iban	pan	pin	cvv	expiring_date
CcS-4857	XX4857591835292505850771	2314242385113924	1819	467	09/27/25
CcS-4858	XX8581768137002436094025	6582720299715533	3964	817	12/28/28
CcS-4859	XX7826930491423553609370	8861684536289642	4983	277	11/26/26
CcS-4860	XX5559590368835304645299	2481155515498459	6876	661	07/27/27
CcS-4861	XX2035182877195191627307	1308930301149557	5710	398	04/25/26

17:05:36 select * from credit_card 5000 row(s) returned

Al crear la tabla con la columna `expiring_date` con el tipo `DATE` el código daba error en el proceso de la carga de los datos desde el archivo “datos_introducir_credit”. Por lo tanto he creado la columna con el tipo `VARCHAR` para después convertirlo a `DATE` con los siguientes pasos:

```
ALTER TABLE credit_card ADD COLUMN temp_date DATE;
UPDATE credit_card
SET temp_date = STR_TO_DATE(expiring_date, '%m/%d/%y');
ALTER TABLE credit_card DROP COLUMN expiring_date;
ALTER TABLE credit_card RENAME COLUMN temp_date TO expiring_date;
```

Paso 3. Hay que relacionar las tablas `credit_card` y `transaction`

El código de abajo se usa para asignar el `credit_card_id` como el Foreign Key en la tabla `credit_card`

```
ALTER TABLE transaction
ADD CONSTRAINT fk_transaction_credit_card
FOREIGN KEY (credit_card_id)
REFERENCES credit_card(id);
```

Paso 4. El diagrama actualizado de la base de datos

Company

Columna	Tipo	Comentario
<u>id</u>	varchar(15) PK	Texto. Primary Key
company_name	varchar(255)	Texto
phone	varchar(15)	Texto
email	varchar(100)	Texto
country	varchar(100)	Texto
website	varchar(255)	Texto

1 N

Transaction

Columna	Tipo	Comentario
<u>id</u>	varchar(255) PK	Texto. Primary Key
<u>credit_card_id</u>	varchar(15)	Texto. Foreign Key
<u>company_id</u>	varchar(20)	Texto. Foreign Key
<u>user_id</u>	int	Número
<u>lat</u>	float	Número
<u>longitude</u>	float	Número
<u>timestamp</u>	timestamp	Tiempo
<u>amount</u>	decimal(10,2)	número redondeado a 2 decimales después de la coma
<u>declined</u>	tinyint(1)	Boolean

N

1

Columna	Tipo	Comentario
<u>id</u>	varchar(15) PK	Texto. Primary Key
iban	varchar(34)	Texto
pan	varchar(19)	Texto
pin	char(4)	Texto
cvv	char(3)	Texto
expiring_date	date	Fecha

Credit_card

La relación entre las tablas es: credit_card (1) - Transaction (N). Están relacionadas a través de la columna credit_card.id (es una Primary Key en la tabla credit_card) y Transaction.credit_card_id (es una foreign key en la tabla Transaction).

La tabla credit_card mantiene la información de las tarjetas de crédito: iban, pan, pin, cvv, la fecha de expiración.

Ejercicio 5

El departamento de Recursos Humanos ha identificado un error en el número de cuenta asociado a su tarjeta de crédito con ID CcU-2938. La información que debe mostrarse para este registro es: TR323456312213576817699999. Recuerda mostrar que el cambio se realizó.

Actualización de los datos:

```
UPDATE transactions.credit_card SET  
credit_card.iban = "TR323456312213576817699999"  
WHERE credit_card.id = "CcU-2938";
```

UPDATE transactions.credit_card SET credit_card.iban = "TR323456312213576817699999" WHERE credit_card.id =... 1 row(s) affected Rows matched: 1 Changed: 1 Warni...

Verificación que los datos han sido actualizados:

```
SELECT * FROM transactions.credit_card  
WHERE credit_card.id = "CcU-2938";
```

id	iban	pan	pin	cvv	expiring_date	
CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	2022-10-30	

SELECT * FROM transactions.credit_card WHERE credit_card.id = "CcU-2938" 1 row(s) returned

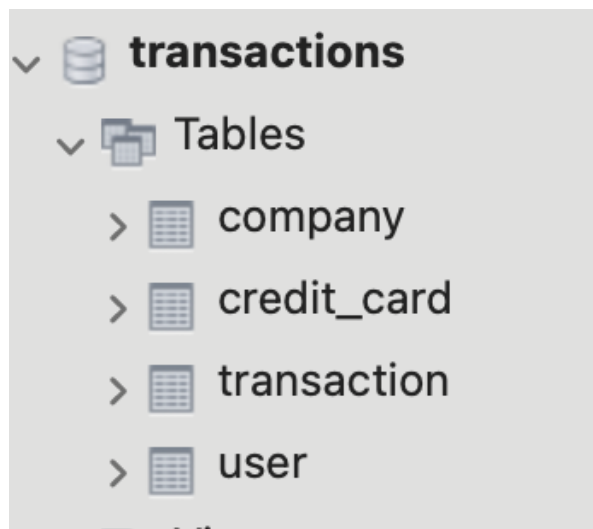
Ejercicio 6

En la tabla "transaction" ingresa una nueva transacción con la siguiente información

Para poder insertar los datos hay que hacer varios pasos previos:

Paso 1. Crear la tabla users ya que la tabla transaction tiene la columna user_id

```
CREATE TABLE IF NOT EXISTS user(  
  id INT PRIMARY KEY  
);
```



Paso 3. En las tablas company, credit_card y users añadir los datos con los ids que salen en el ejercicios. El resto de la información de las correspondientes tablas se puede actualizar más adelante

```
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date)  
VALUES ('CcU-9999', '', '', '', '', '2030-12-31');
```

```
INSERT INTO company (id, company_name, phone, email, country, website)  
VALUES ('b-9999', '', '', '', '', '');
```

```
INSERT INTO user(id)  
VALUES ('9999');
```

Paso 4. Una vez añadidos los datos a las tablas mencionadas arriba se añaden los datos de la tabla transaction

```
INSERT INTO transactions.transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined)
VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', '9999', '829.999', '-117.999', '2015-02-12 02:52:35', '111.11', '0');
```

```
INSERT INTO transactions.transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, dec... 1 row(s) affected
```

Paso 5. Verificación que los datos están añadidos correctamente.

```
SELECT * FROM transactions.transaction
WHERE transaction.id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD';
```

id	credit_card...	company_id	user_id	lat	longitude	timestamp	amount	declined
108B1D1D-5B23-A76C-55EF-C568E49A99DD	CcU-9999	b-9999	9999	829.999	-117.999	2015-02-12 02:52:35	111.11	0

```
SELECT * FROM transactions.transaction WHERE transaction.id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD' 1 row(s) returned
```

Comentario: entre los datos a introducir faltaba el timestamp. En la vida real antes de actualizar la base de datos conseguiría este dato

Ejercicio 7

Desde recursos humanos te solicitan eliminar la columna "pan" de la tabla credit_card. Recuerda mostrar el cambio realizado.

```
ALTER TABLE credit_card DROP COLUMN pan;
```

ALTER TABLE credit_card DROP COLUMN pan

0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

Resultado:

```
SHOW COLUMNS FROM credit_card;
```

Field	Type	Null	Key	Default	Extra
id	varchar(15)	NO	PRI	NULL	
iban	varchar(34)	YES		NULL	
pin	char(4)	YES		NULL	
cvv	char(3)	YES		NULL	
expiring_date	date	YES		NULL	

12:16:40 SHOW COLUMNS FROM credit_card

5 row(s) returned

Ejercicio 8

Descarga los archivos CSV que encontrarás en el apartado de recursos :

american_users.csv

european_users.csv

companies.csv

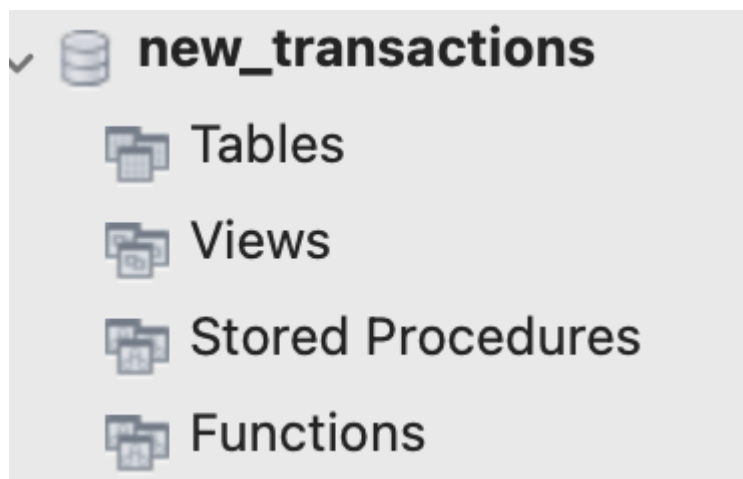
credit_cards.csv

transactions.csv

Estudia y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de las que puedas realizar las siguientes consultas:

Creamos nueva base de datos:

```
CREATE DATABASE IF NOT EXISTS new_transactions;  
USE new_transactions;
```

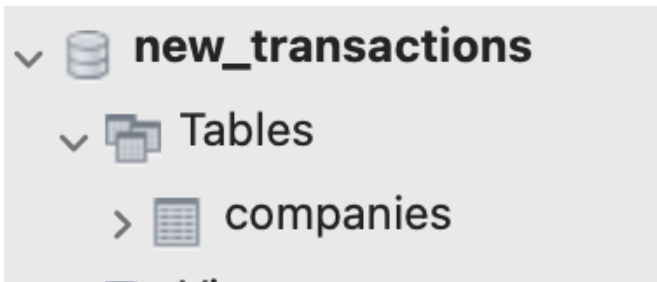


USE new_transactions

0 row(s) affected

Creamos la tabla de empresas:

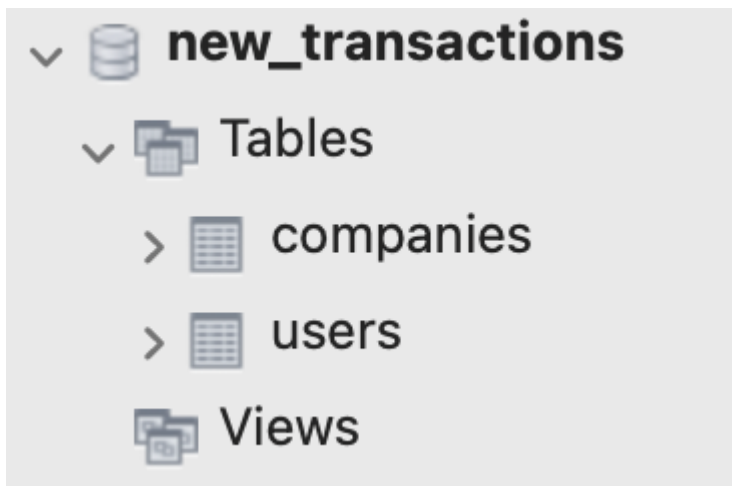
```
CREATE TABLE IF NOT EXISTS companies (  
  id VARCHAR(15) PRIMARY KEY NOT NULL,  
  company_name VARCHAR(255),  
  phone VARCHAR(15),  
  email VARCHAR(100),  
  country VARCHAR(100),  
  website VARCHAR(255)  
);
```



```
CREATE TABLE IF NOT EXISTS companies ( id VARCHAR(15) PRIMARY KEY NOT NULL, company_name VARCHAR(100) NOT NULL ) 0 row(s) affected
```

Creemos la tabla de usuarios a donde añadiremos los datos tanto de los clientes de Europa como de EEUU. Para distinguir que usuarios de que archivo provienen usaremos la columna “region”.

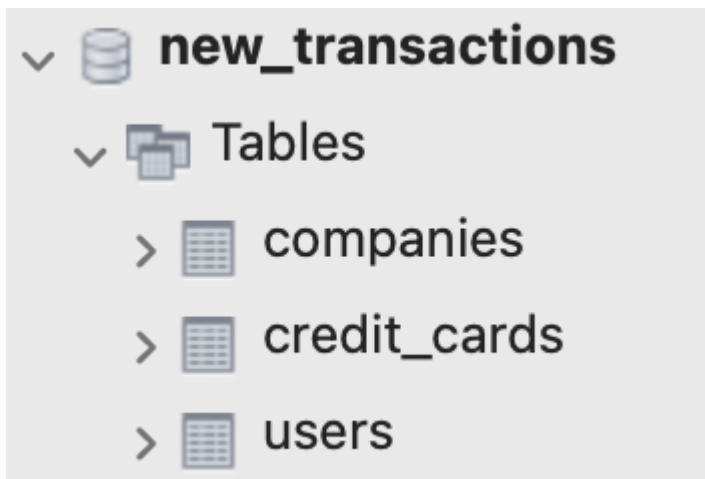
```
CREATE TABLE IF NOT EXISTS users (  
  id INT PRIMARY KEY NOT NULL,  
  name VARCHAR(255),  
  surname VARCHAR(255),  
  phone VARCHAR(15),  
  email VARCHAR(100),  
  birth_date VARCHAR(100),  
  region VARCHAR(100),  
  country VARCHAR(100),  
  city VARCHAR(100),  
  postal_code VARCHAR(100),  
  address VARCHAR(100)  
);
```



CREATE TABLE IF NOT EXISTS users (id INT PRIMARY KEY NOT NULL, name VARCHAR(255), surname VARCH... 0 row(s) affected

Creamos la tabla de tarjetas de crédito.

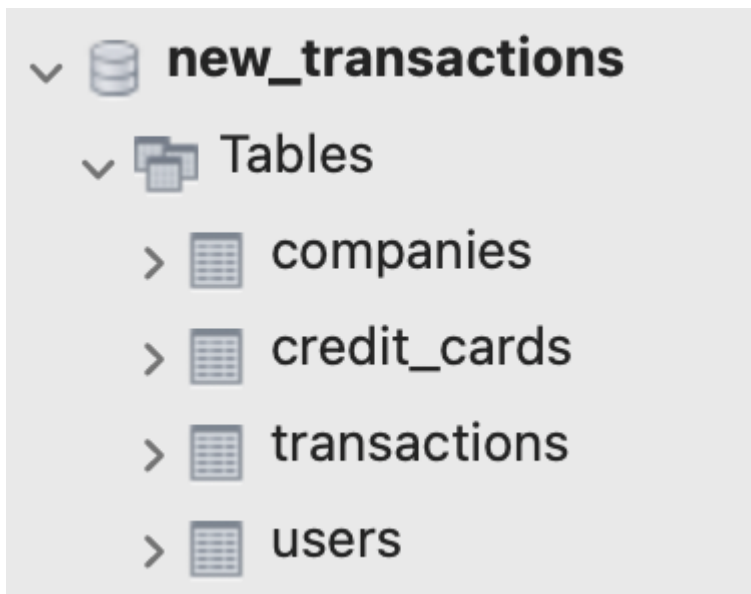
```
CREATE TABLE IF NOT EXISTS credit_cards (  
  id VARCHAR(15) PRIMARY KEY NOT NULL,  
  user_id INT,  
  iban VARCHAR(34),  
  pan VARCHAR(19),  
  pin CHAR(4),  
  cvv CHAR(3),  
  track1 VARCHAR(255),  
  track2 VARCHAR(255),  
  expiring_date VARCHAR(10),  
  CONSTRAINT fk_users  
  FOREIGN KEY (user_id) REFERENCES users (id)  
);
```



```
CREATE TABLE IF NOT EXISTS credit_cards ( id VARCHAR(15) PRIMARY KEY NOT NULL, user_id INT, iban VAR... 0 row(s) affected
```

Creamos la tabla de transacciones.

```
CREATE TABLE IF NOT EXISTS transactions (  
  id VARCHAR(255) PRIMARY KEY NOT NULL,  
  card_id VARCHAR(15),  
  business_id VARCHAR(15),  
  timestamp TIMESTAMP,  
  amount DECIMAL(10, 2),  
  declined BOOLEAN,  
  product_ids VARCHAR(255),  
  user_id INT,  
  lat FLOAT,  
  longitude FLOAT,  
  CONSTRAINT fk_cards  
  FOREIGN KEY (card_id) REFERENCES credit_cards (id),  
  CONSTRAINT fk_companies  
  FOREIGN KEY (business_id) REFERENCES companies (id),  
  CONSTRAINT fk_users_transactions  
  FOREIGN KEY (user_id) REFERENCES users (id)  
);
```



```
CREATE TABLE IF NOT EXISTS transactions ( id VARCHAR(255) PRIMARY KEY NOT NULL, card_id VARCHAR... 0 row(s) affected
```

Cargamos datos en la tabla companies y revisamos que la carga se ejecutó correctamente

```
SET GLOBAL local_infile = 1;
LOAD DATA LOCAL INFILE '/tmp/companies.csv'
INTO TABLE new_transactions.companies
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

```
SELECT * FROM companies;
```

b-2222	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.tellus@yahoo.net	Germany	https://instagram.com/site
b-2226	Magna A Neque Industries	04 14 44 64 62	risus.donec.nibh@icloud.org	Australia	https://whatsapp.com/group/9
b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United States	https://pinterest.com/sub/cars
b-2234	Convallis In Incorporated	06 66 57 29 50	mauris.ut@aol.couk	Germany	https://cnn.com/user/110
b-2238	Ante laoreet Nunc Elevation	00 00 04 00 50	sed.dignam.mauris@outlook.co	New Zealand	https://facebook.com/loremipsum

LOAD DATA LOCAL INFILE '/tmp/companies.csv' INTO TABLE new_transactions.companies FIELDS TERMINATED B... 100 row(s) affected Records: 100 Deleted: 0 Skippe...

SELECT * FROM companies 100 row(s) returned

Cargamos datos en la tabla users. Primero de los usuarios de Europa. A la columna region asignamos “Europe” para saber de donde provienen los usuarios

```
LOAD DATA LOCAL INFILE '/tmp/european_users.csv'
INTO TABLE new_transactions.users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(id, name, surname, phone, email, birth_date, country, city, postal_code, address)
SET region = 'Europe';
```

LOAD DATA LOCAL INFILE '/tmp/european_users.csv' INTO TABLE new_transactions.users FIELDS TERMINATED BY ','... 3990 row(s) affected Records: 3990 Deleted: 0 Skip.
SELECT * FROM new_transactions.users 3990 row(s) returned

id	name	surname	phone	email	birth_date	region	country	city	postal_code	address
151	Meghan	Hayden	0800 746 6747	arcu.vel@hotmail.ca	Jul 2, 1980	Europe	United Kingdom	London	EC1A 1BB	Ap #432-4493 Aliquet Rd.
152	Hakeem	Alford	(0111) 367 0184	adipiscing.ligula@google.edu	Sep 30, 1979	Europe	United Kingdom	Birmingham	B1 1AA	551-8930 Lobortis Street
153	Keegan	Pugh	(016977) 3851	sodales.nisi@aol.org	Jul 27, 1994	Europe	United Kingdom	London	EC1A 1BB	Ap #312-5898 Consectetur St.
154	Cooper	Bullock	(021) 2521 6627	et@outlook.net	Nov 2, 1986	Europe	United Kingdom	Manchester	M1 1AE	872-1866 Pede Rd.

Después hacemos lo mismo con los usuarios de EEUU, indicando el región en el campo correspondiente:

```
LOAD DATA LOCAL INFILE '/tmp/american_users.csv'
INTO TABLE new_transactions.users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(id, name, surname, phone, email, birth_date, country, city, postal_code, address)
SET region = 'USA';

SELECT * FROM new_transactions.users;
```

id	name	surname	phone	email	birth_date	region	country	city	postal_code	address
1	Zeus	Gamble	1-282-581-0551	interdum.enim@protonmail.edu	Nov 17, 1985	USA	United States	New York	10001	348-7818 Sagittis St.
2	Garrett	Mcconnell	(718) 257-2412	integer.vitae.nibh@protonmail.org	Aug 23, 1992	USA	United States	Philadelphia	19101	903 Sit Ave
3	Ciaran	Harrison	(522) 598-1365	interdum.feugiat@aol.org	Apr 29, 1998	USA	United States	Houston	77001	736-2063 Tellus St.

SELECT * FROM new_transactions.users 5000 row(s) returned

Cargamos datos en la tabla credit_cards:

```
LOAD DATA LOCAL INFILE '/tmp/credit_cards.csv'
INTO TABLE new_transactions.credit_cards
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

```
SELECT * FROM new_transactions.credit_cards;
```

id	user_id	iban	pan	pin	cvv	track1	track2	expiring_date
CcS-4857	276	XX4857591835292505850771	2314242385113924	1819	467	%B2314242385113924^LWCBUDLWCBUD^22...	%B2314242385113924=2410101518363164?	09/27/25
CcS-4858	277	XX8581768137002436094025	6582720299715533	3964	817	%B6582720299715533^TIQMVITIQMVV^24040...	%B6582720299715533=2411101104546272?	12/28/28
CcS-4859	278	XX7826930491423553609370	8861684536289642	4983	277	%B8861684536289642^COFBGDCOFBGD^28...	%B8861684536289642=2502101761665371?	11/26/26
CcS-4860	279	XX5559590368835304645299	2481155515498459	6876	661	%B2481155515498459^TIUJTUTIUJTU^31040...	%B2481155515498459=2602101514414395?	07/27/27

LOAD DATA LOCAL INFILE '/tmp/credit_cards.csv' INTO TABLE new_transactions.credit_cards FIELDS TERMINATE... 5000 row(s) affected Records: 5000 Deleted: 0 Skip...
SELECT * FROM new_transactions.credit_cards 5000 row(s) returned

Cargamos datos en la tabla transactions:

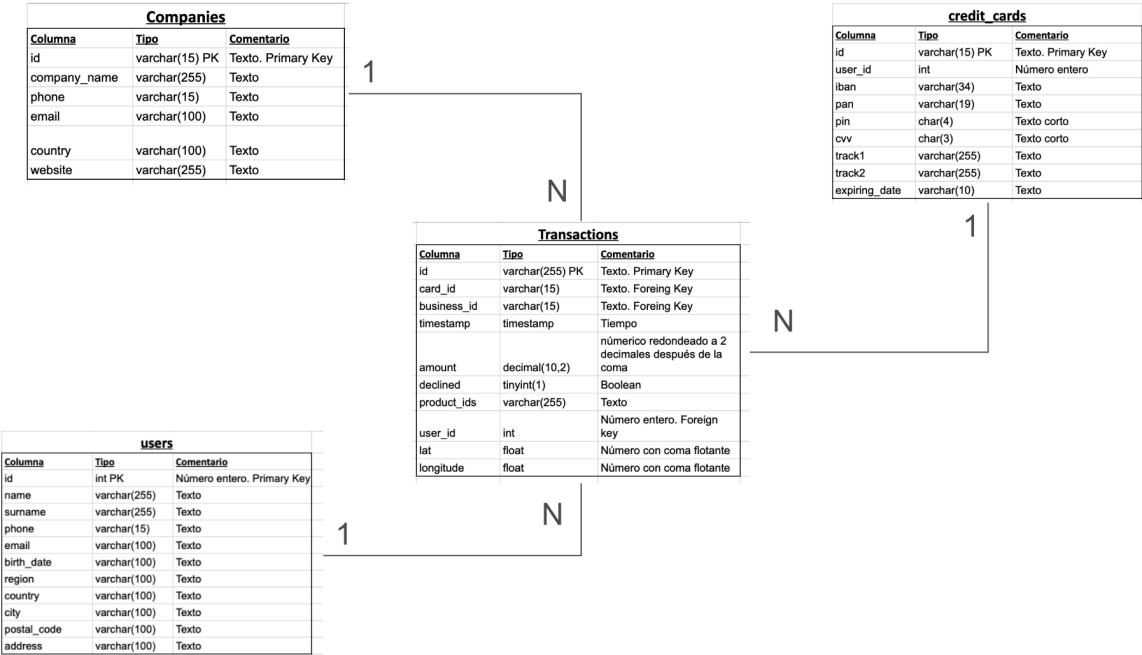
```
LOAD DATA LOCAL INFILE '/tmp/transactions.csv'
INTO TABLE new_transactions.transactions
FIELDS TERMINATED BY ';'
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

```
SELECT * FROM new_transactions.transactions;
```

id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude
00043A49-2949-494B-A5DD-A5BAE3BB19DD	CcS-9294	b-2458	2024-08-28 07:16:46	395.43	0	16, 26, 97, 87	4713	46.1999	1.43554
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	2016-12-21 20:07:18	155.63	0	66, 69, 87	438	41.5972	12.2218
00045D6B-ED2E-4F2F-8186-CEE074D875D0	CcS-6699	b-2390	2020-07-14 15:37:45	326.01	0	30, 11, 16, 81	2118	29.7573	-95.3796
000481C3-1C26-4FEF-83A0-4CD0EB004BBD	CcS-6696	b-2230	2017-09-04 19:44:53	161.60	0	72	2115	53.5489	-113.503

LOAD DATA LOCAL INFILE '/tmp/transactions.csv' INTO TABLE new_transactions.transactions FIELDS TERMINATE... 100000 row(s) affected Records: 100000 Deleted: 0...
SELECT * FROM new_transactions.transactions 100000 row(s) returned

Abajo está el pantallazo del esquema estrella de la base de datos, donde la tabla transactions es la tabla de hechos y las tablas companies, credit_cards y users son las tablas de dimensiones.



Ejercicio 9

Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.

```
SELECT users.id FROM new_transactions.users
JOIN (
    SELECT transactions.user_id, COUNT(transactions.id) as num_trans FROM new_transactions.transactions
    WHERE transactions.declined = 0
    GROUP BY transactions.user_id
) as num_trans_users
ON users.id = num_trans_users.user_id
AND num_trans_users.num_trans >80
;
```

id
185
289
318

```
SELECT users.id FROM new_transactions.users JOIN ( SELECT transactions.user_id, COUNT(transactions.id) as nu... 3 row(s) returned
```

Ejercicio 10

Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd., utiliza por lo menos 2 tablas.

```
SELECT credit_cards.iban, ROUND(AVG(transactions.amount),2) as avg_sales
FROM new_transactions.credit_cards
JOIN new_transactions.transactions
ON credit_cards.id = transactions.card_id
WHERE transactions.business_id = (
    SELECT companies.id FROM new_transactions.companies
    WHERE company_name = 'Donec Ltd'
)
AND transactions.declined = 0
GROUP BY credit_cards.iban
;
```

iban	avg_sales
XX911406401125586307586805	356.25
SK9446370242474562577506	142.96
XX776752917845952975555640	257.37

SELECT credit_cards.iban, ROUND(AVG(transactions.amount),2) as avg_sales FROM new_transactions.credit_cards J... 370 row(s) returned

Nivel 2

Ejercicio 1

Identifica los cinco días que se generó la mayor cantidad de ingresos en la empresa por ventas. Muestra la fecha de cada transacción junto con el total de las ventas.

```
SELECT DATE(transactions.timestamp) AS transaction_date,  
SUM(transactions.amount) AS total_sales  
FROM new_transactions.transactions  
WHERE transactions.declined = 0  
GROUP BY transaction_date  
ORDER BY total_sales DESC  
LIMIT 5;
```

transaction_date	total_sales
2022-12-13	14337.44
2019-11-18	13591.32
2023-02-20	13332.59
2017-12-20	13318.43
2019-03-18	12680.95

```
SELECT companies, companies.company_name, DATE(transactions.timestamp) AS transaction_date, SUM(transactions.amount) AS total_sales FROM new_transactions.transactions  
SELECT DATE(transactions.timestamp) AS transaction_date, SUM(transactions.amount) AS total_sales FROM new_transactions.transactions 5 row(s) returned
```

Ejercicio 2

Presenta el nombre, teléfono, país, fecha y amount, de aquellas empresas que realizaron transacciones con un valor de compras entre 350 y 400 euros y en alguna de estas fechas: 29 de abril de 2015, 20 de julio de 2018 y 13 de marzo de 2024. Ordena los resultados de mayor a menor cantidad.

```
SELECT companies.company_name, companies.phone, companies.country, DATE(transactions.timestamp) as sales_date,
transactions.amount FROM new_transactions.companies
JOIN new_transactions.transactions
ON companies.id = transactions.business_id
WHERE transactions.id IN (
    SELECT transactions.id FROM new_transactions.transactions
    WHERE transactions.amount BETWEEN 350 AND 400
    AND DATE(transactions.timestamp) IN ('2015-04-29', '2018-07-20', '2024-03-13')
)
AND transactions.declined = 0
ORDER BY transactions.amount DESC
;
```

company_name	phone	country	sales_date	amount
Aliquam PC	01 45 73 52 16	Germany	2024-03-13	399.84
Auctor Mauris Vel LLP	08 09 28 74 14	United States	2018-07-20	399.51
At Pede Corp.	06 14 48 33 15	Italy	2015-04-29	390.69

SELECT companies.company_name, companies.phone, companies.country, DATE(transactions.timestamp) as sales_da... 8 row(s) returned

Ejercicio 3

Necesitamos optimizar la asignación de los recursos y dependerá de la capacidad operativa que se requiera, por lo que te piden la información sobre la cantidad de transacciones que realizan las empresas, pero el departamento de recursos humanos es exigente y quiere un listado de las empresas en las que especifiques si tienen igual o más de 400 transacciones o menos.

```
SELECT companies.id, companies.company_name, COUNT(transactions.id),
CASE
  WHEN COUNT(transactions.id) >= 400 THEN '400+'
  ELSE '399-'
END AS num_trans
FROM new_transactions.companies
JOIN new_transactions.transactions
ON companies.id = transactions.business_id
GROUP BY companies.id, companies.company_name;
```

id	company_name	COUNT(transactions....	num_trans
b-2222	Ac Fermentum Incorporated	2401	400+
b-2226	Magna A Neque Industries	410	400+
b-2230	Fusce Corp.	447	400+
b-2234	Convallis In Incorporated	1514	400+
b-2238	Ante Iaculis Nec Foundation	472	400+
b-2242	Deneque Ltd	110	400+

SELECT companies.id, companies.company_name, COUNT(transactions.id), CASE WHEN COUNT(transactions.id) >= 100 row(s) returned

Ejercicio 4

Elimina de la tabla transacción el registro con ID
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de datos.

```
DELETE FROM new_transactions.transactions  
WHERE transactions.id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

```
DELETE FROM new_transactions.transactions WHERE transactions.id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAA... 1 row(s) affected
```

Verificación de que el dato fue borrado correctamente

```
SELECT * FROM new_transactions.transactions  
WHERE transactions.id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

id	card_id	business_id	timestamp	amount	declined	product_i...	user_id	lat	longitude
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
SELECT * FROM new_transactions.transactions WHERE transactions.id = '000447FE-B650-4DCF-85DE-C7ED0EE1CA... 0 row(s) returned
```

Ejercicio 5

La sección de marketing desea tener acceso a información específica para realizar análisis y estrategias efectivas. Se ha solicitado crear una vista que proporcione detalles clave sobre las compañías y sus transacciones. Será necesaria que crees una vista llamada VistaMarketing que contenga la siguiente información: Nombre de la compañía. Teléfono de contacto. País de residencia. Media de compra realizado por cada compañía. Presenta la vista creada, ordenando los datos de mayor a menor promedio de compra.

```
1 • CREATE VIEW `VistaMarketing` AS
2 SELECT companies.company_name, companies.phone, companies.country,
3 ROUND(AVG(transactions.amount),2) AS avg_sale
4 FROM new_transactions.companies
5 JOIN new_transactions.transactions
6 ON companies.id = transactions.business_id
7 AND transactions.declined = 0
8 GROUP BY companies.company_name, companies.phone, companies.country;
```

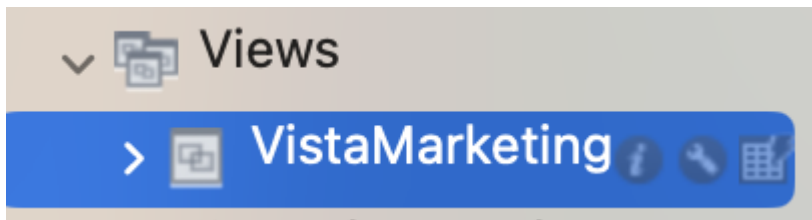
10% 69:8

View

Apply Revert

Apply changes to VistaMarketing

Changes applied



```
SELECT * FROM VistaMarketing
ORDER BY avg_sale DESC;
```

company_name	phone	country	avg_sale
Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.91
Pretium Neque Corp.	07 77 48 55 28	Australia	275.58
Urna Convallis Associates	06 01 24 77 04	United States	273.57
At Associates	09 56 61 10 65	New Zealand	272.74
Metus Vitis Associates	08 85 44 48 66	Australia	270.85

Nivel 3

Ejercicio 1

Crea una nueva tabla que refleje el estado de las tarjetas de crédito basado en si las tres últimas transacciones han sido declinadas entonces es inactivo, si por lo menos una no es rechazada entonces es activo. Partiendo de esta tabla responde:

👉 ¿Cuántas tarjetas están activas?

Creación de la nueva tabla y verificación de los datos cargados

```
CREATE TABLE credit_cards_status AS
SELECT ranked_transactions.card_id,
CASE
    WHEN SUM(ranked_transactions.declined) >= 3 THEN 'Inactive'
    ELSE 'Active'
END AS card_status
FROM (
    SELECT transactions.card_id, transactions.id, transactions.declined, RANK()
    OVER (PARTITION BY transactions.card_id ORDER BY DATE(transactions.timestamp) DESC)
    AS rank_transaction_date
    FROM new_transactions.transactions
) AS ranked_transactions
WHERE rank_transaction_date <= 3
GROUP BY card_id;
SELECT * FROM new_transactions.credit_cards_status;
```

▼ **new_transactions**

Tables

>  companies

>  credit_cards

>  credit_cards_status

>  transactions

>  users



>  Views

Stored Procedures

	card_id	card_status	
	CcS-4867	Active	
	CcS-4868	Active	
	CcS-4869	Active	
	CcS-4870	Inactive	
	CcS-4871	Active	
	CcS-4872	Active	

```
CREATE TABLE credit_cards_status AS SELECT ranked_transactions.card_id, CASE WHEN SUM(ranked_transaction... 5000 row(s) affected Records: 5000 Duplicates: 0...
SELECT * FROM credit_cards_status 5000 row(s) returned
```

Hay 4995 tarjetas activas

```
SELECT COUNT(*) active_cards FROM new_transactions.credit_cards_status
WHERE credit_cards_status.card_status = 'Active';
```

active_cards
4995

```
SELECT COUNT(*) active_cards FROM new_transactions.credit_cards_status WHERE credit_cards_status.card_status... 1 row(s) returned
```